

STL $\rightarrow$ Iterator (Pointer) $\rightarrow$ Iteration $\rightarrow$ STL
(Template)

① Input Iterator

② Output Iterator

③ Forward Iterator $\checkmark \rightarrow$ Every Template $\rightarrow$ Forward Direction

④ Bi-Directional Iterator $\left\{ \begin{array}{l} \rightarrow \text{Forward + Backward} \rightarrow \text{Sequence + Associative} \\ \rightarrow \text{Forward Direction} \rightarrow \text{Unordered Container} \end{array} \right.$

⑤ Random Iterator $\left[\begin{array}{l} \rightarrow f \\ \rightarrow B \\ \rightarrow R \end{array} \right. \rightarrow \left[\begin{array}{l} \rightarrow \text{Array} \\ \rightarrow \text{Vector} \\ \rightarrow \text{Deque} \end{array} \right]$

① Forward Iterator:- With the help of Pointer we will access the element one by one in a forward Direction.

• STL Template

- Vector
- list
- Set
- map (K, V)

→ Every Template

Note:- Pointer:- Address Store

• Targeted Variable →
Data_type Ptx

• (*)Ptx → To access Value
"asterisk"

Vector < Char > v1 = { 'A'₂₀₀₀, 'g'₂₀₀₁, 'D'₂, 'e'₃, 'F'₄ }

auto itr⁽²⁰⁰⁰⁾ = v1.begin();

While (²⁰⁰⁰itr != ²⁰⁰⁵v1.end())
{

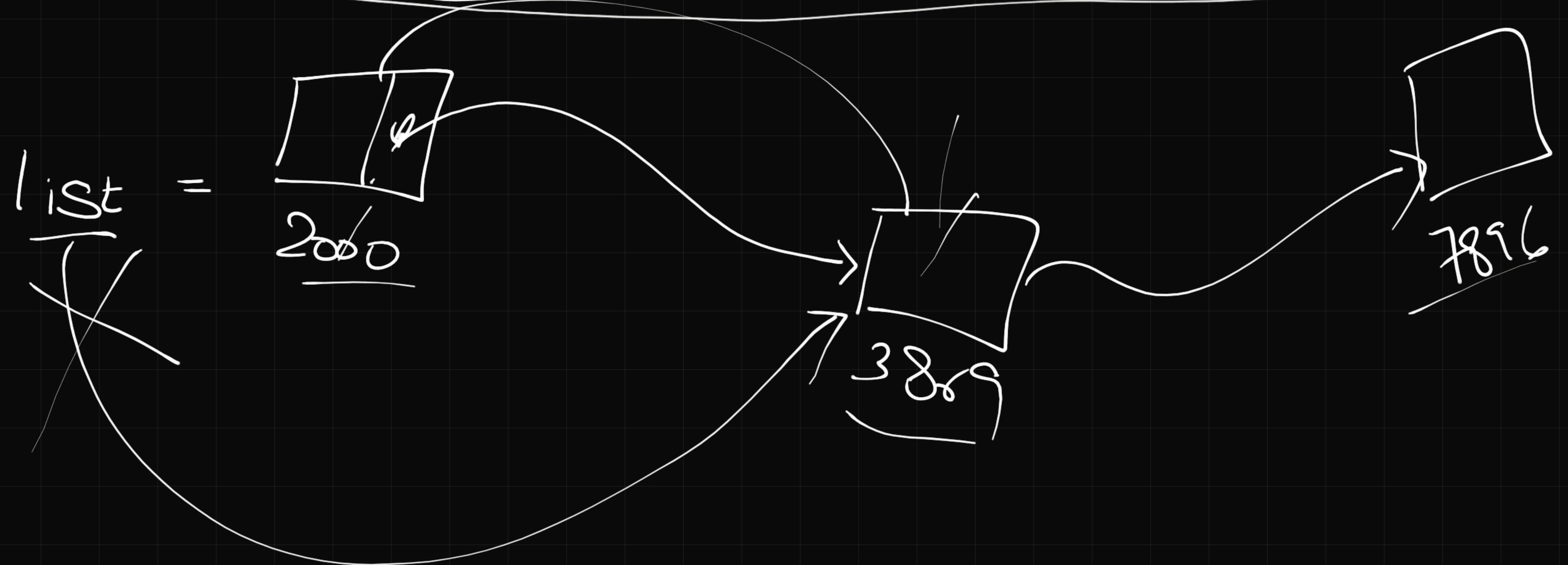
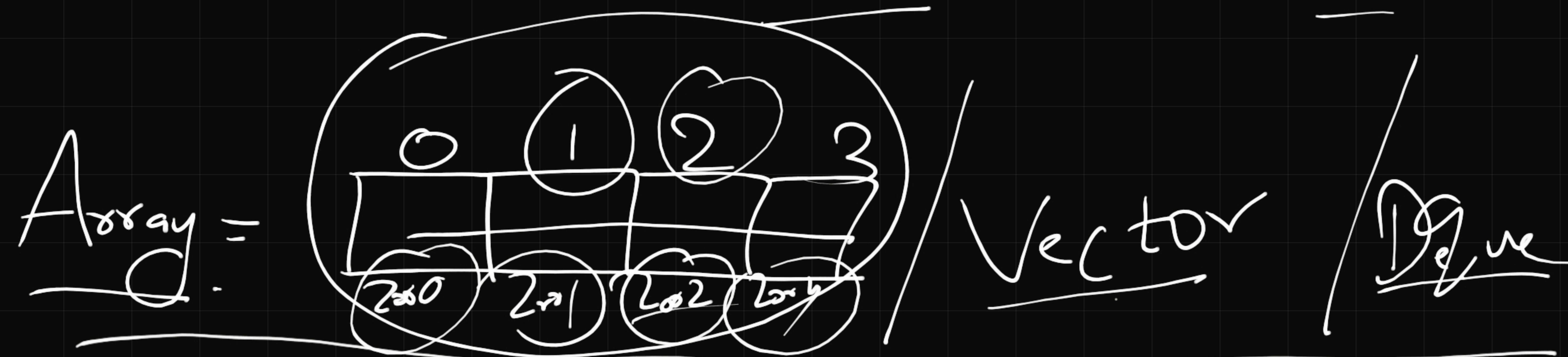
-- / ++itr;
} cout << *itr << endl; itr = (itr + 1)

2000 + 1
= 2001

② B. $\rightarrow$ Directional Iterators $\rightarrow$ With the help of a pointer
We will access the element
one by one but in both
direction "forward / Backward"

Note: - Except Unordered Containers $\rightarrow$ $\left(\begin{array}{c} \text{Backward} \\ \text{Direction} \end{array} \right)$
X

③ Random Iterator - Random index $\rightarrow$ Value



list =
(2068)

